

Algoritmos Avançados

Aulas 08 & 09: Busca Sequencial & POO (Atributos e Métodos)

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário

- 1 O Problema da Busca e a Busca Sequencial
- 2 Introdução à Orientação a Objetos (POO)
- 3 Classes, Atributos e o Construtor
- 4 Métodos de Instância e Métodos Especiais
- 5 Integração: Busca Sequencial em Coleções de Objetos
- 6 Exercícios Práticos Resolvidos

O Problema da Busca em Computação

Definição Formal

Dada uma coleção de elementos $C = [c_0, c_1, c_2, \dots, c_{n-1}]$ e uma chave de busca K (alvo), o problema da busca consiste em:

- Determinar se $K \in C$; e/ou
- Encontrar o índice i tal que $c_i = K$; ou
- Retornar todas as ocorrências de K na coleção.

Aplicações Práticas

- Consulta a registros de sensores agrícolas.
- Identificação de vértices em grafos.
- Filtragem em tabelas de Big Data.

Desafio Central

Como localizar a informação com o **menor custo computacional** possível quando o volume de dados cresce?

O que é a Busca Sequencial (Linear Search)?

É o algoritmo de busca mais elementar e intuitivo:

- Inicia no primeiro elemento da coleção (índice 0).
- Compara o elemento atual com a chave de busca desejada.
- Se for igual: a busca encerra com sucesso (**sucesso**).
- Se for diferente: avança sequencialmente para a próxima posição.
- Se atingir o fim da coleção sem encontrar o elemento: a busca encerra com fracasso (**falha**).

Premissa Fundamental

A Busca Sequencial **não exige que os dados estejam ordenados**. Ela funciona sobre qualquer sequência arbitrária.

Busca Sequencial: Implementação Básica (Índice)

A função abaixo percorre a lista e retorna o índice onde o alvo foi encontrado, ou -1 se não existir:

```
1 def busca_sequencial(lista: list, alvo: any) -> int:
2     for i in range(len(lista)):
3         if lista[i] == alvo:
4             return i    # Parada imediata ao encontrar
5     return -1          # Percorreu toda a lista e nao encontrou
6
7 # Teste em colecao de leituras
8 temperaturas = [28.5, 31.0, 24.2, 33.1, 29.8]
9 pos = busca_sequencial(temperaturas, 24.2)
10 print(f'Encontrado no indice: {pos}')    # Output: 2
11
12 pos_inexistente = busca_sequencial(temperaturas, 40.0)
13 print(f'Resultado busca: {pos_inexistente}')    # Output: -1
```

Busca Sequencial: Retornando Booleano

Caso o objetivo seja apenas saber se o dado existe na coleção:

```
1 def contem_elemento(lista: list, alvo: any) -> bool:
2     for item in lista:
3         if item == alvo:
4             return True # Encerra antecipadamente
5     return False
6
7 culturas = ['Soja', 'Milho', 'Algodao', 'Cafe']
8 print(contem_elemento(culturas, 'Milho')) # True
9 print(contem_elemento(culturas, 'Laranja')) # False
```

Conexão com o Python Nativo

O operador `in` do Python (`'Milho' in culturas`) executa internamente uma busca linear similar sobre a estrutura da lista.

Busca Sequencial: Múltiplas Ocorrências

Quando a chave pode aparecer repetidas vezes e precisamos de todas as posições:

```
1 def busca_todas_ocorrencias(lista: list, alvo: any) -> list[int]:
2     indices = []
3     for i, item in enumerate(lista):
4         if item == alvo:
5             indices.append(i)
6     return indices
7
8 falhas_sensores = [0, 1, 0, 0, 1, 1, 0, 1]
9 posicoes_falha = busca_todas_ocorrencias(falhas_sensores, 1)
10 print(f'Falhas nos indices: {posicoes_falha}') # [1, 4, 5, 7]
```

Atenção ao Critério de Parada

Neste cenário, **não há interrupção antecipada**; a varredura completa até o final da lista é obrigatória.

Busca Sequencial com Sentinela (Otimização Clássica)

Podemos eliminar o teste de limite de índice ($i < \text{len}(\text{lista})$) em cada passo inserindo o alvo no final:

```
1 def busca_com_sentinela(lista: list, alvo: any) -> int:
2     n = len(lista)
3     lista.append(alvo)    # Adiciona o elemento sentinela no final
4     i = 0
5     while lista[i] != alvo:
6         i += 1
7     lista.pop()           # Remove o sentinela para restaurar a lista
8     return i if i < n else -1
9
10 dados = [15, 30, 45, 60, 75]
11 print(busca_com_sentinela(dados, 45))    # 2
12 print(busca_com_sentinela(dados, 99))    # -1
```

Ganho Computacional

Reduz de duas comparações por iteração (limite e igualdade) para apenas uma.

Busca Sequencial em Lista Ordenada

Se a lista estiver previamente ordenada, podemos interromper a busca antes:

```
1 def busca_sequencial_ordenada(lista: list[int], alvo: int) -> int:
2     for i, valor in enumerate(lista):
3         if valor == alvo:
4             return i
5         elif valor > alvo:
6             # Elemento atual maior que o alvo: nenhum posterior sera igual!
7             break
8     return -1
9
10 ordenados = [10, 25, 40, 55, 70, 85, 100]
11 print(busca_sequencial_ordenada(ordenados, 30)) # -1 (parou no 40)
```

Análise de Complexidade da Busca Sequencial

A eficiência da busca depende da posição onde o elemento se encontra:

Melhor Caso: $O(1)$

O alvo está na **primeira posição** (índice 0).

- Apenas 1 comparação executada.

Caso Médio: $O(n)$

O alvo está distribuído aleatoriamente na lista.

- Em média $\frac{n+1}{2}$ comparações.

Pior Caso: $O(n)$

O alvo está na **última posição** ou **inexiste**.

- Realiza exatamente n comparações.

Complexidade Espacial (Memória): $O(1)$ – algoritmo in-place.

Busca Sequencial em Arquivos de Texto (Stream)

Em Big Data, datasets massivos muitas vezes não cabem inteiros na memória RAM. A busca sequencial permite processamento em fluxo (*streaming*):

```
1 def buscar_registro_arquivo(caminho_arquivo: str, id_alvo: str) -> str |  
   None:  
2     # Leitura linha a linha com memória O(1) constante!  
3     with open(caminho_arquivo, 'r', encoding='utf-8') as f:  
4         for num_linha, linha in enumerate(f, start=1):  
5             if id_alvo in linha:  
6                 return f'Linha {num_linha}: {linha.strip()}'  
7     return None
```

Vantagem em Ciência de Dados

Permite analisar logs ou arquivos de telemetria de dezenas de gigabytes com uso mínimo de memória.

Vantagens e Limitações da Busca Sequencial

Vantagens

- Algoritmo direto e sem pré-requisitos.
- Opera sobre dados **desordenados**.
- Excelente para coleções pequenas ($n \leq 100$).
- Ideal para fluxos lineares e arquivos sequenciais.

Limitações em Big Data

- Custo linear $O(n)$ inaceitável para milhões de consultas repetidas.
- Não aproveita ordenação ou estruturas de índice.
- Dá origem a estruturas avançadas: **Busca Binária** ($O(\log n)$) e **Tabelas Hash** ($O(1)$).

Limitações da Abordagem Procedural / Estruturada

Na programação estruturada comum, **dados e funções ficam desacoplados**:

```
1 # Abordagem Estruturada: variaveis soltas e funcoes genericas
2 talhao_1_id = 'T-01'
3 talhao_1_area = 120.5
4 talhao_1_prod = 85.0
5
6 def calcular_producao(area: float, prod_por_ha: float) -> float:
7     return area * prod_por_ha
8
9 total = calcular_producao(talhao_1_area, talhao_1_prod)
```

Problemas com o crescimento do software

- Dificuldade de manter a integridade dos dados (qualquer trecho do programa pode alterar atributos indevidamente).
- Falta de coesão: dados e as regras de negócio que operam sobre eles estão dispersos.

O Paradigma Orientado a Objetos (POO)

A **POO** propõe organizar o sistema em torno de **objetos** que modelam entidades reais, unindo **estado (dados)** e **comportamento (funções)** na mesma estrutura.

Estado (Atributos)

O que o objeto **é** ou o que ele **possui**.

- Identificador, nome, área, capacidade.
- Valores numéricos, textos, coleções.

Comportamento (Métodos)

O que o objeto **faz** ou como ele reage.

- Calcular produção total.
- Validar leitura de sensor.
- Conectar a um vértice vizinho.

Classe vs. Objeto: A Distinção Central

Definition (Classe)

É o **molde**, a planta baixa ou a especificação técnica que descreve as características e comportamentos que os objetos daquele tipo possuirão.

Definition (Objeto (Instância))

É a **materialização concreta** da classe criada na memória durante a execução do programa.

Analogia Construtiva

- A planta de engenharia é a **Classe** Trator.
- O trator real com chassi #9872 operando na lavoura é o **Objeto**.

Os Pilares da POO (Visão Geral)

A Orientação a Objetos é sustentada por quatro pilares conceituais:

Abstração & Encapsulamento

- **Abstração:** Focar nas características essenciais, ignorando detalhes supérfluos.
- **Encapsulamento:** Ocultar o estado interno e restringir acessos indevidos.

Herança & Polimorfismo

- **Herança:** Reutilizar código criando especializações a partir de classes base.
- **Polimorfismo:** Tratar diferentes tipos derivados sob uma mesma interface comum.

Por que POO em Algoritmos Avançados?

Ao longo desta disciplina, construiremos estruturas com alto grau de abstração:

- **Nós de Listas e Árvores:** Instâncias da classe Node que guardam uma carga útil e referências para outros nós.
- **Grafos:** Objetos Grafo gerenciando coleções de objetos Vertice e Aresta.
- **Pilhas e Filas:** TADs encapsulados onde a manipulação interna é blindada contra acessos indevidos.

Conclusão Prática

A POO fornece as ferramentas estruturadas para implementar os modelos matemáticos e abstratos da nossa ementa.

Definindo uma Classe Básica em Python

Utiliza-se a palavra-chave `class` seguida pelo nome em **PascalCase** (PEP 8):

```
1 class Talhao:
2     pass # Instrucao nula temporaria
3
4 # Instanciando dois objetos distintos da classe Talhao
5 t1 = Talhao()
6 t2 = Talhao()
7
8 print(t1)           # <__main__.Talhao object at 0x7f8b90...>
9 print(type(t1))     # <class '__main__.Talhao'>
10 print(t1 is t2)     # False (alocados em enderecos de memoria diferentes!)
```

O Método Construtor `__init__`

O método especial `__init__` é executado **automaticamente** no momento em que um novo objeto é instanciado:

```
1 class Talhao:
2     def __init__(self, identificador: str, area_ha: float, cultura: str):
3         # Atributos de Instancia (Pertencem ao objeto especifico)
4         self.identificador = identificador
5         self.area_ha = area_ha
6         self.cultura = cultura
7
8     # Criacao de objetos passando os argumentos necessarios
9 t1 = Talhao('T-01', 150.0, 'Soja')
10 t2 = Talhao('T-02', 85.5, 'Milho')
11
12 print(f'Talhao: {t1.identificador} | Cultura: {t1.cultura}')
13 print(f'Talhao: {t2.identificador} | Area: {t2.area_ha} ha')
```

O que é o parâmetro self?

Entendendo o self

- self representa a **referência à própria instância** do objeto que invocou o método.
- Permite que cada objeto mantenha seu próprio estado isolado dos demais.
- O interpretador Python passa o objeto **automaticamente** como primeiro argumento:

`t1.calcular()` $\iff$ `Talhao.calcular(t1)`

Regra de Sintaxe

Todo método de instância em Python deve declarar self obrigatoriamente como seu primeiro parâmetro formal.

Atributos de Instância vs. Atributos de Classe

Atributos de Instância

- Definidos via `self.atributo`.
- Cada objeto possui sua própria cópia.

Atributos de Classe

- Definidos fora de qualquer método.
- **Compartilhados** por todas as instâncias.

```
1 class Sensor:
2     PROTOCOLO = 'LoRaWAN'  # Compartilhado por todos os sensores
3
4     def __init__(self, id_sensor: int, bateria: float):
5         self.id_sensor = id_sensor  # Exclusivo de cada objeto
6         self.bateria = bateria      # Exclusivo de cada objeto
7
8 s1 = Sensor(101, 95.0)
9 s2 = Sensor(102, 80.0)
10 print(s1.PROTOCOLO, s2.PROTOCOLO)  # LoRaWAN LoRaWAN
```

Como o Python Armazena Atributos: O `__dict__`

Internamente, objetos Python armazenam seus atributos de instância em um dicionário:

```
1 class Talhao:
2     def __init__(self, cod: str, area: float):
3         self.cod = cod
4         self.area = area
5
6 t = Talhao('T-1', 50.0)
7 print(t.__dict__)    # {'cod': 'T-1', 'area': 50.0}
8
9 # O Python permite atribuir novos atributos dinamicamente em tempo de
   execucao:
10 t.responsavel = 'Douglas'
11 print(t.__dict__)    # {'cod': 'T-1', 'area': 50.0, 'responsavel': 'Douglas'}
```

Boa Prática em Algoritmos

Evite criar atributos dinâmicos fora do `__init__` para manter o código previsível e consistente.

Otimização de Memória com `__slots__`

Em Big Data, ao instanciar milhões de nós ou registros, o dicionário `__dict__` consome muita RAM. O uso de `__slots__` elimina esse overhead:

```
1 class VerticeOtimizado:
2     # Define previamente apenas os atributos permitidos, sem usar __dict__
3     __slots__ = ['id_vertice', 'peso']
4
5     def __init__(self, id_vertice: int, peso: float):
6         self.id_vertice = id_vertice
7         self.peso = peso
8
9 v = VerticeOtimizado(1, 4.5)
10 # Economiza de 40% a 50% de espaço em memória ao instanciar grandes colecoes
    !
```

Métodos de Instância (Comportamento)

Funções que operam sobre os atributos internos do objeto através de self:

```
1 class Talhao:
2     def __init__(self, id_talhao: str, area_ha: float, prod_esperada: float)
3         :
4         self.id_talhao = id_talhao
5         self.area_ha = area_ha
6         self.prod_esperada = prod_esperada # sacas / ha
7
8     def calcular_producao_total(self) -> float:
9         return self.area_ha * self.prod_esperada
10
11     def atualizar_produtividade(self, novo_valor: float) -> None:
12         if novo_valor > 0:
13             self.prod_esperada = novo_valor
14
15 talhao = Talhao('T-Norte', 100.0, 75.0)
16 print(f'Producao Total: {talhao.calcular_producao_total():.1f} sacas') #
17     7500.0
```

Representação Textual: O Método `__str__`

Por padrão, exibir um objeto com `print()` mostra seu endereço de memória. Sobrescrever `__str__` fornece uma saída amigável:

```
1 class Sensor:
2     def __init__(self, id_sensor: str, valor: float, unidade: str):
3         self.id_sensor = id_sensor
4         self.valor = valor
5         self.unidade = unidade
6
7     def __str__(self) -> str:
8         # Chamado por print() e str()
9         return f'Sensor [{self.id_sensor}]: {self.valor:.1f} {self.unidade}'
10
11 s = Sensor('TEMP-01', 27.85, 'C')
12 print(s) # Sensor [TEMP-01]: 27.9 C
```

Depuração e Logs: O Método `__repr__`

Enquanto `__str__` é voltado para o usuário final, `__repr__` é voltado para o desenvolvedor e para exibição em coleções:

```
1 class Sensor:
2     def __init__(self, id_sensor: str, valor: float):
3         self.id_sensor = id_sensor
4         self.valor = valor
5
6     def __repr__(self) -> str:
7         return f'Sensor(id={self.id_sensor!r}, valor={self.valor})'
8
9 lista_sensores = [Sensor('S1', 20.5), Sensor('S2', 25.0)]
10 # Ao imprimir a lista inteira, o Python invoca o __repr__ de cada elemento:
11 print(lista_sensores)
12 # Output: [Sensor(id='S1', valor=20.5), Sensor(id='S2', valor=25.0)]
```

Igualdade Customizada: O Método `__eq__`

Por padrão, o operador `==` compara ponteiros de memória. Com `__eq__`, comparamos o conteúdo dos atributos:

```
1 class Vertice:
2     def __init__(self, rotulo: str):
3         self.rotulo = rotulo
4
5     def __eq__(self, outro: object) -> bool:
6         if isinstance(outro, Vertice):
7             return self.rotulo == outro.rotulo
8         return False
9
10 v1 = Vertice('A')
11 v2 = Vertice('A')
12 print(v1 is v2)    # False (dois objetos fisicos distintos)
13 print(v1 == v2)    # True! (Comportamento garantido pelo metodo __eq__)
```

Ordenação de Objetos: O Método `__lt__`

Sobrescrever `__lt__` (*less than*) habilita operadores relacionais (`<`) e funções de ordenação direta (`sort()`, `sorted()`):

```
1 class Talhao:
2     def __init__(self, id_talhao: str, area: float):
3         self.id_talhao = id_talhao
4         self.area = area
5
6     def __lt__(self, outro: 'Talhao') -> bool:
7         return self.area < outro.area # Critério: menor area vem primeiro
8
9     def __repr__(self) -> str:
10        return f'Talhao({self.id_talhao}, {self.area}ha)'
11
12 fazenda = [Talhao('T3', 200.0), Talhao('T1', 50.0), Talhao('T2', 120.0)]
13 fazenda.sort()
14 print(fazenda) # [Talhao(T1, 50.0ha), Talhao(T2, 120.0ha), Talhao(T3, 200.0
ha)]
```

Encapsulamento e Propriedades (@property)

Garante que os atributos sejam validados e controlados sem perder a sintaxe natural de acesso:

```
1 class RegistroSolo:
2     def __init__(self, ph: float):
3         self._ph = None
4         self.ph = ph # Dispara o setter para validacao
5
6     @property
7     def ph(self) -> float:
8         return self._ph
9
10    @ph.setter
11    def ph(self, valor: float) -> None:
12        if not (0.0 <= valor <= 14.0):
13            raise ValueError('0 valor de pH deve estar entre 0 e 14.')
14        self._ph = valor
15
16 amostra = RegistroSolo(6.5)
17 print(f'pH validado: {amostra.ph}') # 6.5
18 # amostra.ph = 18.0 -> Dispara ValueError com seguranca!
```

Relações entre Classes: Composição vs. Herança

Em modelagem de algoritmos e TADs, a relação entre entidades define a arquitetura do programa:

Composição (Relação Tem-Um)

Uma classe contém instâncias de outra classe como atributos.

- Um **Grafo** *tem* uma lista de **Vértices**.
- Uma **Fazenda** *tem* vários **Talhões**.
- Favorece baixo acoplamento e flexibilidade.

Herança (Relação É-Um)

Uma classe filha especializa e herda comportamentos da classe mãe.

- Um **SensorUmidade** *é um* **Sensor**.
- Um **GrafoPonderado** *é um* **Grafo**.

Buscando Objetos por Atributo Específico

Na prática, buscamos instâncias de classes em coleções comparando um atributo-chave (identificador):

```
1 class DispositivoAgro:
2     def __init__(self, serial: str, status: str):
3         self.serial = serial
4         self.status = status
5
6 def buscar_dispositivo(colecao: list[DispositivoAgro], serial_alvo: str) ->
    DispositivoAgro | None:
7     for disp in colecao:
8         if disp.serial == serial_alvo:
9             return disp      # Retorna a referencia ao proprio objeto encontrado
10    return None               # Retorna None caso nao exista
11
12 dispositivos = [DispositivoAgro('S-10', 'Ativo'), DispositivoAgro('S-20', '
    Falha')]
13 resultado = buscar_dispositivo(dispositivos, 'S-20')
14 if resultado:
15     print(f'Localizado! Status atual: {resultado.status}')
```



Busca Sequencial com Critérios Dinâmicos (Lambdas)

Podemos criar uma função de busca genérica que aceita qualquer critério booleano (*predicado*):

```
1 from typing import Callable
2
3 def buscar_primeiro(colecao: list[any], predicado: Callable[[any], bool]) ->
4     any | None:
5     for item in colecao:
6         if predicado(item):
7             return item
8     return None
9
10 sensores = [
11     Sensor('S1', 22.0),
12     Sensor('S2', 39.5), # Temperatura critica!
13     Sensor('S3', 24.1)
14 ]
15 # Localiza o primeiro sensor com temperatura acima de 35 graus
16 critico = buscar_primeiro(sensores, lambda s: s.valor > 35.0)
17 print(f'Alerta no dispositivo: {critico}')
```



Busca Sequencial Elegante com next()

O Python oferece uma forma concisa de realizar busca sequencial com iteradores e expressões geradoras:

```
1 dispositivos = [  
2     DispositivoAgro('S-01', 'Inativo'),  
3     DispositivoAgro('S-02', 'Ativo'),  
4     DispositivoAgro('S-03', 'Ativo')  
5 ]  
6  
7 # next(gerador, valor_padrao_se_nao_encontrar)  
8 alvo = next((d for d in dispositivos if d.serial == 'S-02'), None)  
9  
10 if alvo:  
11     print(f'Encontrado via next(): {alvo.serial} -> {alvo.status}')
```

```
12 else:  
13     print('Nao localizado.')
```

Exercício Prático 1: Enunciado

Problema 1: Monitoramento de Umidade do Solo

Modele uma classe `SensorUmidade` que possua os atributos `id_sensor` (str) e `leitura_pct` (float, entre 0% e 100%).

- 1 Valide a leitura de umidade no momento da instanciação.
- 2 Escreva uma função de busca sequencial `buscar_sensor_seco(sensores, limite_minimo)` que retorne o primeiro sensor cuja umidade esteja abaixo do limite aceitável de irrigação.

Exercício Prático 1: Solução Comentada

```
1 class SensorUmidade:
2     def __init__(self, id_sensor: str, leitura_pct: float):
3         if not (0.0 <= leitura_pct <= 100.0):
4             raise ValueError('Leitura deve estar entre 0 e 100%.')
5         self.id_sensor = id_sensor
6         self.leitura_pct = leitura_pct
7
8     def __repr__(self) -> str:
9         return f'Sensor({self.id_sensor}: {self.leitura_pct:.1f}%)'
10
11 def buscar_sensor_seco(sensores: list[SensorUmidade], limite: float) ->
    SensorUmidade | None:
12     for s in sensores:
13         if s.leitura_pct < limite:
14             return s
15     return None
16
17 rede = [SensorUmidade('U-01', 45.0), SensorUmidade('U-02', 18.5),
18         SensorUmidade('U-03', 52.0)]
19 critico = buscar_sensor_seco(rede, 20.0)
```



Exercício Prático 2: Enunciado

Problema 2: Mapeamento de Vértices de um Grafo

Para prepararmos a estrutura que utilizaremos em Grafos:

- 1 Modele a classe `Vertice` contendo `id_vertice` (int) e uma lista de vizinhos adjacentes.
- 2 Implemente o método `adicionar_vizinho(vizinho)` evitando duplicatas.
- 3 Implemente uma busca sequencial que verifique se dois vértices possuem conexão direta entre si (se um é adjacente ao outro).

Exercício Prático 2: Solução Comentada

```
1 class Vertice:
2     def __init__(self, id_vertice: int):
3         self.id_vertice = id_vertice
4         self.adjacentes: list['Vertice'] = []
5
6     def adicionar_vizinho(self, vizinho: 'Vertice') -> None:
7         if vizinho not in self.adjacentes:
8             self.adjacentes.append(vizinho)
9
10    def possui_aresta_para(self, id_alvo: int) -> bool:
11        for v in self.adjacentes:
12            if v.id_vertice == id_alvo:
13                return True    # Conexao direta confirmada
14        return False
15
16 v1 = Vertice(1)
17 v2 = Vertice(2)
18 v1.adicionar_vizinho(v2)
19 print(v1.possui_aresta_para(2))    # True
20 print(v1.possui_aresta_para(3))    # False
```



Exercício Prático 3: Enunciado

Problema 3: Catálogo de Talhões e Busca por Cultura

Uma cooperativa agrícola armazena uma lista de objetos Talhao.

- 1 Implemente a função `buscar_talhoes_por_cultura(catalogo, cultura_alvo)` que retorne todos os talhões que plantam a referida cultura.
- 2 Calcule e imprima a área total somada desses talhões localizados.

Exercício Prático 3: Solução Comentada

```
1 def buscar_talhoes_por_cultura(catalogo: list[Talhao], cultura_alvo: str) ->
    list[Talhao]:
2     resultado = []
3     for t in catalogo:
4         if getattr(t, 'cultura', None) == cultura_alvo:
5             resultado.append(t)
6     return resultado
7
8 # Simulacao de registros
9 propriedade = [
10     Talhao('T-01', 100.0, 75.0),
11     Talhao('T-02', 45.0, 60.0),
12     Talhao('T-03', 120.0, 80.0)
13 ]
14 propriedade[0].cultura = 'Soja'
15 propriedade[1].cultura = 'Milho'
16 propriedade[2].cultura = 'Soja'
17
18 talhoes_soja = buscar_talhoes_por_cultura(propriedade, 'Soja')
19 area_total_soja = sum(t.area ha for t in talhoes_soja)
```



Próximos Passos

Conteúdo da Próxima Aula (Aula 10)

- **Algoritmos de Ordenação – Bubblesort (Ordenação Bolha)**
- Princípio de trocas adjacentes e propagação de máximos.
- Implementação iterativa do Bubblesort em Python.
- Otimização com flag de interrupção precoce e análise assintótica $O(n^2)$.

Tarefa Complementar

Implementar a modelagem de classes e os algoritmos de busca no laboratório prático.

Bons Estudos! Dúvidas ou Perguntas?